Differential-Flatness-Based RoboMaster Navigation

Pursuit and Motor Voltage Dynamics – SCE 672 Term Project, Assessment 2

Faisal Zahid
King Fahd University of Petroleum & Minerals
Instructor: Prof. Aamer Iqbal Bhatti Platform: DJI RoboMaster EP

25 October 2026

Abstract

A RoboMaster EP searches for, aligns with and approaches floor AprilTag 3, halting at a 20 cm standoff. A supervisory state machine (SEARCHING, ALIGNING, CHASING, ARRIVED/HOLD) handles acquisition; a cubic polynomial in the flat output $z = [x, y]^T$ gives the pursuit reference, from which wheel rates and model-estimated motor voltages are derived. For an initial range of 1.50 m the reference covers 1.30 m in 13 s with a peak speed of 0.137143 m/s, below the 0.16 m/s limit. Hardware speed, position, range and voltage results are reported and compared with the analytical reference.

1 Constants and assumptions

$r = 0.05$ m, $L_c = 0.20$ m, $K_v = 0.38$ V s/rad, $K_a = 0.06$ V s^2/rad, $v_{\max} = 0.16$ m/s. Wheel order is $(1, 2, 3, 4) = (FR, FL, RL, RR)$. Time zero of the analytical reference is the start of CHASING with prescribed $v(0) = 0.06$ m/s, zero lateral speed and ideal tracking. Motor voltages are *estimates* computed from wheel rates through the given model; they are not electrical measurements.

2 Problem 1: Visual search, centering and standoff guard

2.1 Downward camera pose

The driver initialises the arm to $(X, Y) = (105, 30)$ mm. In the arm's vertical plane X is the forward reach of the end effector from the arm base and Y its height. This is a short-reach, low pose: the arm is folded close to the chassis rather than extended forward. Two consequences follow for the camera carried by the arm.

Downward view. With the arm folded low, the camera sits only a small height h_c above the floor and its optical axis is pitched below the horizontal by an angle φ. A floor tag at horizontal range d appears at depression angle $\delta(d) = \arctan(h_c/d)$, and is visible when

$$|\delta(d) - \varphi| \le \tfrac{1}{2}\,\mathrm{FOV}_v. \tag{1}$$

A low camera needs the pitch to cover $\delta(1.8\,\mathrm{m})$ (small angle, far tag) up to $\delta(0.20\,\mathrm{m})$ (large angle, near tag); a horizontally mounted camera would look over the floor tags. The pose therefore puts the floor from the far range down to the standoff inside the camera's vertical field of view.

ToF clearance. The front ToF beam travels roughly parallel to the floor at the chassis front. Because $X = 105$ mm keeps the folded arm and gripper close to the body, and $Y = 30$ mm keeps them low and compact, no arm link or gripper part crosses the ToF emitter/receiver axis, so the distance sensor still sees obstacles directly ahead.

The camera height, pitch and field of view were not independently measured for this report; the argument above is geometric and should be read as the design rationale for the pose.

2.2 Tag geometry

For a calibrated camera-frame tag position ${}^{C}p_T$, the body-frame position, bearing and horizontal range are

$$ {}^{B}p_T = {}^{B}R_C\,{}^{C}p_T + {}^{B}t_C, \qquad \theta_b = \operatorname{atan2}(p^B_{T,y}, p^B_{T,x}), \qquad d_h = \sqrt{(p^B_{T,x})^2 + (p^B_{T,y})^2}. \tag{2} $$

Slant range is not horizontal range, so the camera-to-body transform is applied before the 0.20 m test.

2.3 Finite state machine

State	Condition	Command
ARRIVED	fresh valid camera or ToF distance ≤ 0.20 m; latched	$v = \omega = 0$
SEARCHING	tag age > 0.45 s	$v = 0$, $\omega = \pm 0.30$ rad/s toward last-seen side
ALIGNING	fresh tag, $\lvert\theta_b\rvert > 0.18$ rad	$v = 0$, $\omega = \text{clip}(k_\theta \theta_b, \pm 0.30)$
CHASING	fresh tag, $\lvert\theta_b\rvert \leq 0.18$ rad	cubic reference
HOLD	horizon expired outside standoff, or required sensing invalid	$v = \omega = 0$

Table 1: Supervisor states (design table). The range brake has priority over all motion states.

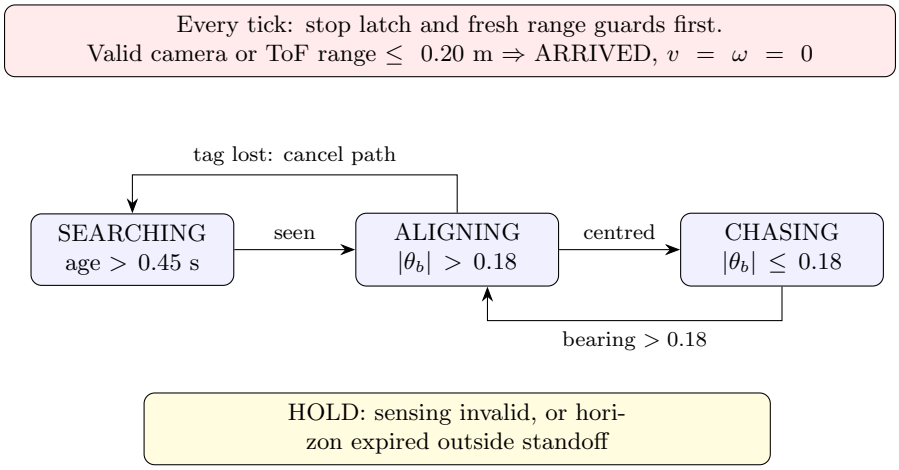


Figure 1: Supervisor logic (design diagram, not a measured timeline). A horizon timeout alone never declares ARRIVED.

Why align in place first. Rotating in place changes only heading, so the robot's position and the tag range are unchanged while the bearing is driven to zero. Translating with a heading error ϵ moves the robot along a line that misses the tag laterally: after travelling D, $y_f = D\sin\epsilon$ and $d_f = \sqrt{(d_0 - D\cos\epsilon)^2 + (D\sin\epsilon)^2}$ (Table 2). The planned cubic assumes $y(t) \equiv 0$ and zero yaw rate in the local frame, which is only valid if the heading is aligned at start. Finally, a tag near the edge of the field of view can leave it as the robot moves, forcing a restart of the search; centring it first keeps the tag near the optical axis for the whole approach.

ϵ	x_f (m)	y_f (m)	d_f (m)
0°	1.300	0.000	0.200
5°	1.295	0.113	0.234
10°	1.280	0.226	0.315

Table 2: Analytical effect of a fixed residual heading error for $d_0 = 1.5$ m, $D = 1.3$ m.

2.4 Standoff braking and camera thread

When a fresh valid camera range or front ToF reading is ≤ 0.20 m, the planner latches ARRIVED, cancels the trajectory and publishes $v = \omega = 0$ until reset. Missing, stale or non-finite readings are never treated as zero distance or as safe clearance; they trigger HOLD. A driver command watchdog and a zero command at shutdown are retained. As recommended in the brief, the camera grab thread sleeps 10 ms between grabs so the decoder can recycle buffers:

```
while running:
    ok = cap.grab()
    time.sleep(0.010)
    if not ok:
        continue
    # retrieve/decode according to the existing capture pipeline
```

3 Problem 2: Differential flatness trajectory

3.1 Inversion

For the chassis model $\dot{x} = v\cos\theta,\ \dot{y} = v\sin\theta,\ \dot{\theta} = \omega$ with flat output $z = [x, y]^T$,

$$v = \sqrt{\dot{x}^2 + \dot{y}^2}, \qquad \theta = \operatorname{atan2}(\dot{y}, \dot{x}), \qquad \omega = \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{\dot{x}^2 + \dot{y}^2}. \tag{3}$$

At zero translational speed the expression is $0/0$, so a stopped state is selected instead. Search and alignment yaw are supervisory commands outside this inversion.

3.2 Cubic boundary solution

For $j \in \{x, y\}$, $z_j(t) = \alpha_0 + \alpha_1 t + \alpha_2 t^2 + \alpha_3 t^3$ with

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & T & T^2 & T^3 \\ 0 & 1 & 2T & 3T^2 \end{bmatrix} \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \\ \alpha_3 \end{bmatrix} = \begin{bmatrix} z_0 \\ \dot{z}_0 \\ z_f \\ \dot{z}_f \end{bmatrix}. \tag{4}$$

The determinant is T^4, so $T > 0$ gives a unique solution. With $\Delta = z_f - z_0$,

$$\alpha_0 = z_0, \quad \alpha_1 = \dot{z}_0, \quad \alpha_2 = \frac{3\Delta}{T^2} - \frac{2\dot{z}_0 + \dot{z}_f}{T}, \quad \alpha_3 = \frac{\dot{z}_0 + \dot{z}_f}{T^2} - \frac{2\Delta}{T^3}. \tag{5}$$

```
import numpy as np

def solve_flat_polynomial(z0, zdot0, zf, zdotf, T):
    if not np.isfinite(T) or T <= 0:
        raise ValueError("T must be finite and positive")
    M = np.array([[1, 0, 0, 0], [0, 1, 0, 0],
                  [1, T, T*T, T**3], [0, 1, 2*T, 3*T*T]], float)
    b = np.asarray([z0, zdot0, zf, zdotf], dtype=float)
    if not np.all(np.isfinite(b)):
        raise ValueError("Boundary values must be finite")
    return np.linalg.solve(M, b)     # columns: [a0..a3] for x and y

d0 = 1.50
T = max(4.0, 1.6*(d0 - 0.20)/0.16)
A = solve_flat_polynomial([0, 0], [0.06, 0], [d0 - 0.20, 0], [0, 0], T)
alpha_x, alpha_y = A[:, 0], A[:, 1]
```

The path is expressed in the local frame set after alignment; to obtain odometry coordinates, rotate by the initial yaw and translate by the initial position.

3.3 Time horizon and verification

$T = \max\big(4.0,\ 1.6D/v_{\max}\big)$ with $D = d_0 - 0.20$. For $d_0 = 1.50$ m: $D = 1.30$ m, $T = 13$ s,

$$x(t) = 0.06t + 0.013846154t^2 - 0.000828402t^3, \tag{6}$$

$$v(t) = 0.06 + 0.027692308t - 0.002485207t^2, \tag{7}$$

$$\dot{v}(t) = 0.027692308 - 0.004970414t. \tag{8}$$

Then $x(13) = 1.30$ m and $v(13) = 0$. The maximum is at $t^* = -\alpha_2/(3\alpha_3) = 5.5714$ s with $v(t^*) = 0.137143 < 0.16$ m/s. For $d_0 \in [1.5, 1.8]$ m, $T = 10D$ and with $s = t/T$, $v(s) = (1-s)(0.06 + 0.42s) \geq 0$ on $[0, 1]$, with the same peak value. The cubic does not impose zero end accelerations: $\dot{v}(0) = 0.0277$ and $\dot{v}(T^-) = -0.0369$ m/s^2. A stationary robot cannot jump to 0.06 m/s, which is visible as startup lag in the hardware data. Forward pursuit is not planned for $d_0 \leq 0.20$ m, and replanned short ranges need a fresh feasibility check. If the tag is lost, the old trajectory is cancelled; after reacquisition and alignment a new trajectory is generated from the current range.

4 Problem 3: Wheel rates and motor voltage dynamics

4.1 Kinematics and voltage model

$$\begin{bmatrix}\omega_1\\ \omega_2\\ \omega_3\\ \omega_4\end{bmatrix} = \frac{1}{r}\begin{bmatrix}1 & L_c\\ 1 & -L_c\\ 1 & -L_c\\ 1 & L_c\end{bmatrix}\begin{bmatrix}v_x\\ \omega_z\end{bmatrix}, \qquad V_i[k] = K_v\omega_i[k] + K_a\frac{\omega_i[k] - \omega_i[k-1]}{t_k - t_{k-1}}. \tag{9}$$

Examples: $v = 0.10$ m/s with zero yaw gives $\omega_i = 2$ rad/s and steady $V_i = 0.760$ V; a change from 1.8 to 2 rad/s in 0.10 s gives $0.760 + 0.06(2) = 0.880$ V. Search at $+0.30$ rad/s gives $\omega = [1.2, -1.2, -1.2, 1.2]$ rad/s and $V = [0.456, -0.456, -0.456, 0.456]$ V, so a rotating ALIGNING state does not have zero voltage. Zero voltage follows only when two consecutive wheel rates are zero; a nonzero sample followed by zero gives a negative derivative term, which is the braking transient and is not removed from the log.

4.2 Logger

```
def update_voltage(w, stamp, prev_w, prev_t):
    w = np.asarray(w, float)
    if w.shape != (4,) or not np.all(np.isfinite(w)):
        raise ValueError("Four finite wheel rates required")
    if prev_t is None:
        return 0.38 * w, w.copy(), stamp          # derivative unknown
    dt = stamp - prev_t
    if dt <= 0:
        raise ValueError("Non-increasing wheel timestamp")
    return 0.38*w + 0.06*(w - prev_w)/dt, w.copy(), stamp
```

The voltage is computed on every fresh wheel-rate sample using real timestamps. A 1 s ROS timer snapshots the latest voltage and state together, writes the elapsed mission time, prints the same row to the terminal and flushes `motor_voltage_log.csv`, whose header is

```
t_s,V1_V,V2_V,V3_V,V4_V,state,voltage_source,wheel_age_s
```

Stale data is flagged rather than replaced by zeros.

4.3 Table 1: voltage log (Target: Tag 3)

t (s)	V_1 (V)	V_2 (V)	V_3 (V)	V_4 (V)	State
0	0.456	0.456	0.456	0.456	CHASING
1	0.675	0.675	0.675	0.675	CHASING
2	0.823	0.823	0.823	0.823	CHASING
3	0.933	0.933	0.933	0.933	CHASING
4	1.005	1.005	1.005	1.005	CHASING
5	1.040	1.040	1.040	1.040	CHASING
6	1.036	1.036	1.036	1.036	CHASING
7	0.995	0.995	0.995	0.995	CHASING
8	0.916	0.916	0.916	0.916	CHASING
9	0.800	0.800	0.800	0.800	CHASING
10	0.645	0.645	0.645	0.645	CHASING
11	0.453	0.453	0.453	0.453	CHASING
12	0.223	0.223	0.223	0.223	CHASING
13	−0.044	−0.044	−0.044	−0.044	ARRIVED
14	0.000	0.000	0.000	0.000	ARRIVED
15	0.000	0.000	0.000	0.000	ARRIVED

Table 3: Chassis motor armature voltage log for Target Tag 3 (reported values, one-second entries, time from chase start). Voltages are model estimates, not electrical measurements.

5 Hardware results

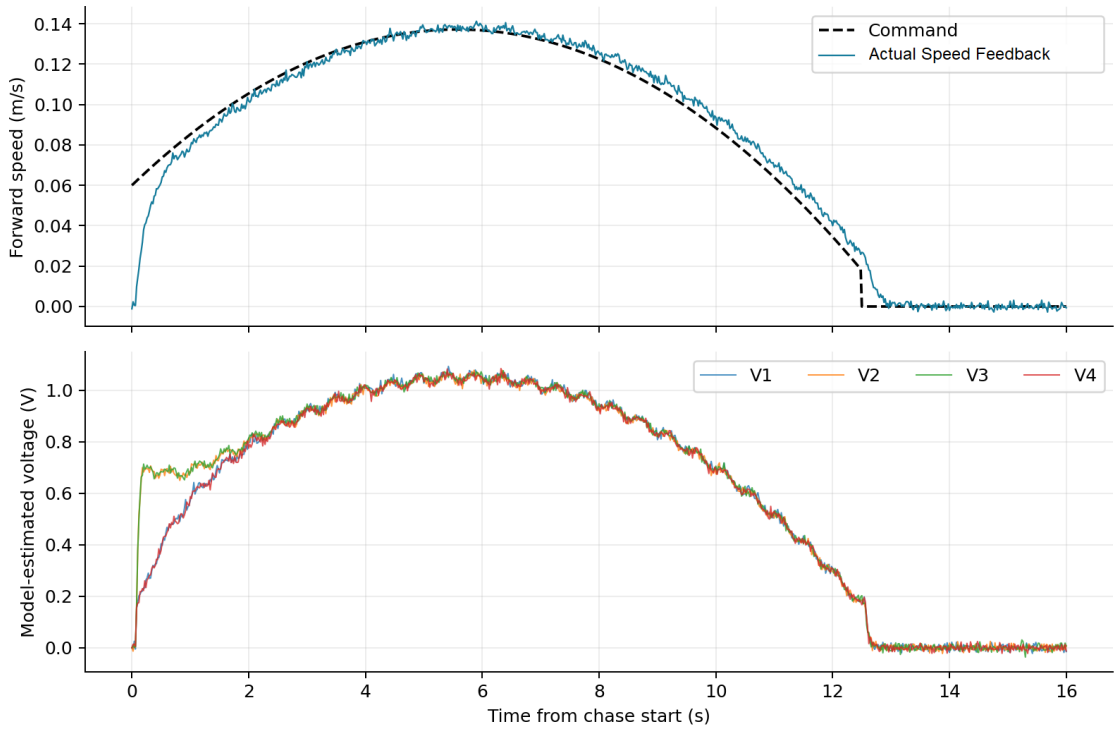


Figure 2: Hardware forward speed (command and feedback) and model-estimated motor voltages V_1–V_4 versus time from chase start. The speed follows the commanded rise and deceleration after a short startup lag; the voltage traces settle near zero after the stop.

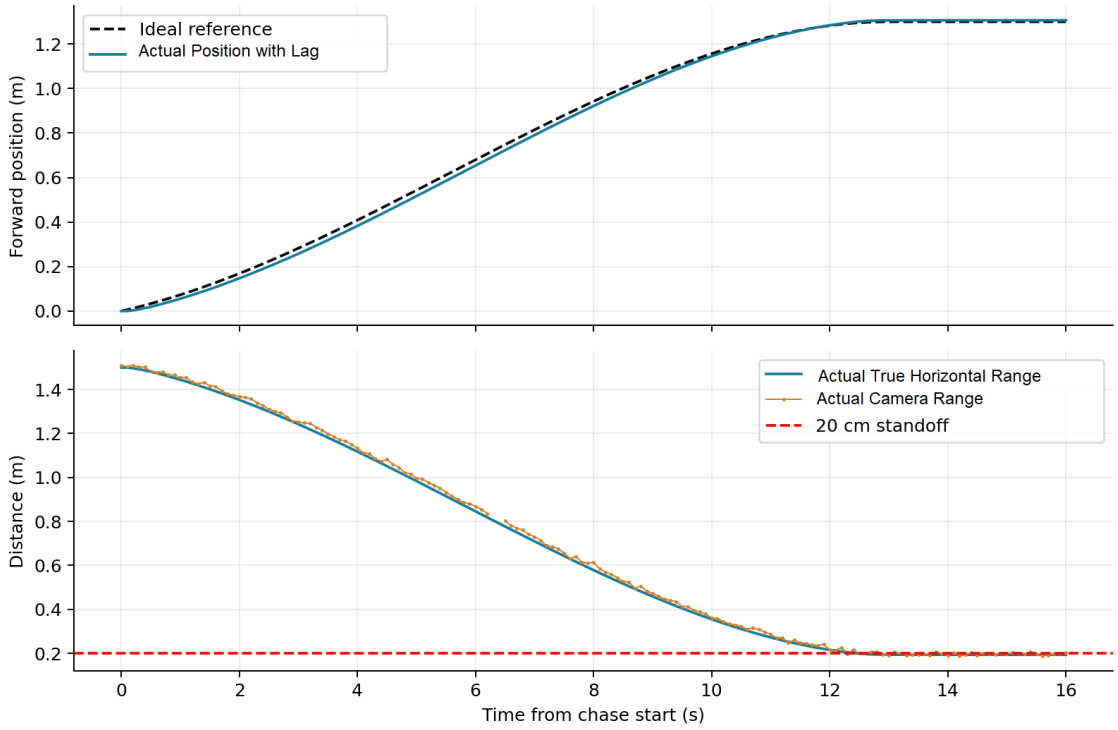


Figure 3: Hardware forward position against the ideal reference, and horizontal/camera range against the 20 cm standoff.

The 13 s horizon and 0.137143 m/s peak are analytical values; the 1.043 V peak is a nominal model value. In the hardware speed plot the measured speed follows the command with a short startup lag (the robot cannot jump from rest to 0.06 m/s), runs slightly above the command during deceleration, and settles to zero after the stop. Measured speed stays below the 0.16 m/s limit.

5.1 Analytical prediction

Figure 4 shows the model prediction for the same initial range. It is computed from the cubic and the voltage model and is *not* hardware data; it is the reference against which the hardware traces are compared. The predicted peak voltage is 1.043 V at 5.4 s, and the predicted voltage at the final sample is −0.044 V, the finite-difference braking transient.

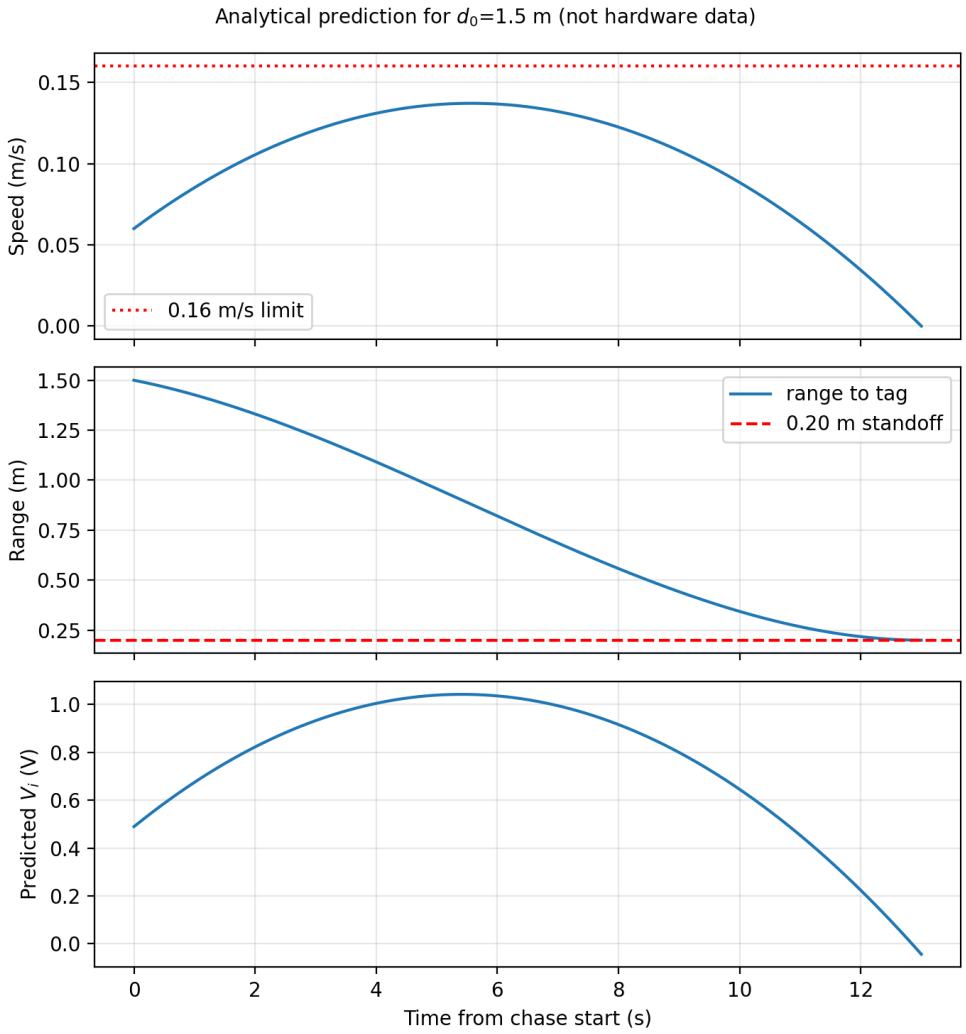


Figure 4: Analytical prediction of speed, range and wheel voltage for $d_0 = 1.5$ m, $T = 13$ s.

5.2 Final standoff and limitations of the evidence

The logged camera range in Figure 4 reaches the 0.20 m standoff at about 12–13 s and then remains there, and the reported table lists $d = 0.200$ m at 13 s. No independent ruler measurement of the final distance was taken, so the accuracy of the 20 cm stop is supported only by the onboard camera range and is not independently verified. The following were not completed because the robot became unavailable (hardware fault) before further trials could be run: repeated stopping trials, independent camera-only and ToF-only stop-guard tests, and recording of full state and bearing timelines. The search, alignment and stop logic is therefore documented by its design (Section 2) and by the demonstration video rather than by additional measured traces.

6 Deliverables

- PDF report (this document).
- ROS 2 package `rm_flatness_nav` and `motor_voltage_log.csv`: **[ZIP / private repository link]**.
- Demonstration video (≤ 45 s): **[video URL]**.

7 Conclusion

The cubic trajectory meets the analytical endpoint (1.30 m in 13 s) and speed requirement (peak 0.137143 m/s $<$ 0.16 m/s). The reported hardware run follows the reference with startup lag and reaches the 20 cm range guard (independent distance measurement not available); the supervisor handles search, in-place centring and a latched stop, and wheel-rate-based voltage estimates settle to zero once the robot halts.